Learning Verified Monitors for Hidden Markov Models

Luko van der Maas and Sebastian Junges

Radboud University, Nijmegen, the Netherlands

Abstract. Runtime monitors for hidden Markov models obtain a sequence of observations and must classify whether the system is safe or unsafe. That is, the probability of being in a dangerous state conditioned on these observations exceeds some acceptable threshold. The key contribution of this paper is the first correct-by-construction synthesis method for such monitors, represented as finite state machines. First, we establish the coNP-hardness of checking whether a finite state machine is indeed a correct monitor, i.e., a monitor without misclassifications. Second, we provide a reduction that reformulates the search for misclassifications into a standard probabilistic system synthesis problem. Third, we integrate the verification routine into an active automata learning routine to synthesise correct monitors. Fourth, we provide a prototypical implementation that shows the feasibility and limitations of the approach on a series of benchmarks.

1 Introduction

Runtime assurance is an essential ingredient in the deployment of safe autonomous systems. Runtime monitors provide assurance by flagging potentially dangerous system behavior by observing a system execution. More precisely, a monitor receives a stream of observations about the system and outputs a verdict, e.g., it raises an alarm that the system has left some safety envelope. A monitor is correct, if it correctly raises such alarms based on a formal specification. Various challenges in creating correct runtime monitors for (semi-)autonomous systems have been identified [**empty citation**], such as: (1) the state of the system is only partially observable, i.e., the stream of observations comes from sensor readings and does uniquely identify the state of a system, (2) the behavior of the system and/or the sensors may be subject to probabilistic uncertainty, (3) the monitor itself is subject to resource constraints (time, memory, etc), and (4) the monitor is itself safety-critical and should therefore be subject to extensive validation. Challenges (1,2) can be (partially) addressed by modelling the system as a hidden Markov model (HMM), Challenges (3,4) can be (partially) addressed by representing a monitor as, e.g., a (small) finite state machine. Concretely, this paper focuses on the following question: *Is a given finite state machine a correct monitor for a given HMM?* The paper studies the complexity of this problem, provides a practical verification procedure, and embeds this verification procedure into a learning framework.

What are HMMs? In this paper, we assume that the system including its sensors is adequately described as a discrete Hidden Markov Model (HMM). Markov chains (MCs) describe system behavior subject to probabilistic uncertainty. Paths through an MC are sequences of states that describe system executions. HMMs extend MCs by labelling their states with colours (aka *observations*). Intuitively, the observations can be used to model the information that the monitor receives. In HMMs, every path can be lifted to a sequence of observations, which we call a *trace*. The trace associated to a system execution describes the information received by the monitor.

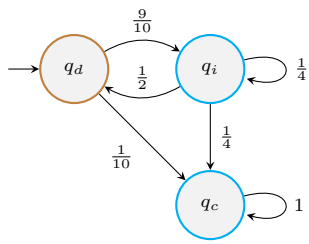
Risk of a trace. In runtime monitoring, an essential task is to estimate whether the current system is dangerous. In the simplest setting, a specification describes a set of states is dangerous. The risk of a trace is then the probability of ending in a dangerous state, conditioned on the fact that the system execution matches the trace. For a given trace, we may compute this risk, e.g., either via a reduction to model checking [**empty citation**] or by a (forward) filtering that tracks a distribution over the current states [**empty citation**].

What are correct monitors? Often, we are not interested in this precise risk, but rather want to partition the set of horizon-bounded traces into safe and unsafe traces: Unsafe traces are traces whose risk does exceed some fixed *acceptable threshold*, all other traces are safe. Indeed, a *correct* monitor obtains a trace and raises an alarm iff a trace is not safe. The key verification problem in this paper asks whether a monitor, represented as a deterministic finite automaton, accepts (i.e., raises an alarm) exactly the unsafe traces. We note that in this paper, we only consider traces that are bounded by some fixed horizon.

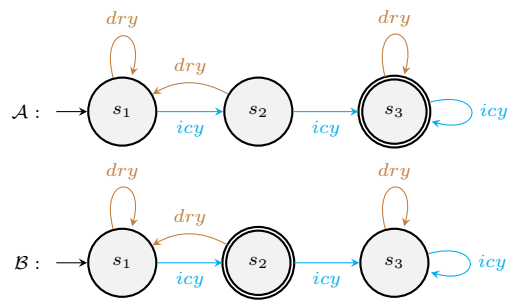
Illustrative toy example. SJ: TODO make more concrete, add Fig. As an running example, we consider an oversimplified car driving scenario, loosely inspired by runtime monitors obtained from high-fidelity simulations [**empty citation**]. A car can be in three states: It can be on a dry road, on an icy patch, or it can drifted off the road. The HMM in Figure 3 describes the situation, where a car alternates between dry and icy road segments, and where being on a icy (dry) segment positively affects the probability that the next segment is icy (dry). When on an icy road, there is a higher probability to drift off the road. Our sensor can detect dry roads, but cannot distinguish between icy roads and off-road conditions. We can define the trace risk as the probability to be off-road, conditioned on a a trace, such as $dry \cdot ice \cdot ice$. In **??**, we depict a finite automaton that describes the set of traces that end with two consecutive icy patches. The key verification question in this paper is now: Is this a correct monitor for threshold 0.9? Put differently, does it raise an alarm iff the trace risk is above 0.9.

SJ: Do we also want a small figure of a system feeding observations into a monitor?

Novel complexity results. Deciding whether a monitor is correct is coNP-complete (Theorem 2). In particular, we study the dual problem that asks whether a monitor misclassifies at least one trace. We consider a simpler variant of this problem that asks whether there is a bounded trace in the HMM that is unsafe.



(a) Example HMM, where the cyan states q_i and q_c have the observation *icy* and the brown state q_d has the observation *dry*.



(b) Two example monitors for the HMM in fig. 1a, correct monitor $\mathcal{A}$ and faulty monitor $\mathcal{B}$. $\mathcal{A}$ raises an alarm as soon as two steps on an *icy* section have been observed. While $\mathcal{B}$ raises an alarm after exactly one step on icy road has been observed.

Fig. 1: An HMM and a correct monitor that raises an alarm if the probability of being off-road is above 0.8.

[1] The problem is clearly in NP (assuming that the bound is unary encoded). We show that this simplified problem is also strongly NP-hard (Lemma 1) and the corresponding optimization problem, that asks to approximate the risk of the trace with the heighest risk, is APX-hard (Lemma 2). Intuitively, APX-hardness means that the existence of an efficient approximation algorithm would have major complexity-theoretic implications.

Our verification approach. While deciding whether a monitor is correct is thus in general not tractable, we suggest to utilise recent advances in the synthesis of probabilistic systems. We concentrate on proving the absence of missed alarms, i.e., we concentrate on showing that the monitor correctly identifies every unsafe trace. A similar reduction works for showing the absence of spurious alarm. First, for a single trace, computing the risk can be reduced to computing a reachability probability in a MC that is intuitively a kind of product between a DFA that accepts exactly the trace and the HMM [**empty citation**]. Thus, it is natural to reduce our problem to the question: Is there a DFA (accepting exactly one trace, accepted by our monitor) such that the probability in the product MC exceeds a threshold. If the answer is no, then the monitor has no missed alarms. This decision problem can be solved using (exact) probabilistic model synthesis meth-

[1] The results also apply to checking for safe traces.

TODO add our framework

Fig. 2

ods [**empty citation**] as implemented in the tool Paynt [**empty citation**][2]. We discuss our verification approach in Section 3.

Our learning approach. Ultimately, we do not only want to verify the correctness of a monitor, but we want to construct such a monitor. Above, we already mentioned that the monitors are formal languages. When considering bounded trace, these monitors are regular, and can thus be captured using DFAs. Inspired by a recent short paper [**empty citation**], we investigate learning these monitors: We choose to do this using active automata learning [3, 8], which closely relates to other popular oracle-guided inductive synthesis loops [**empty citation**]. To use active automata learning (AAL), we must provide a *membership oracle* that decides for a single trace whether the risk of the trace exceeds a threshold and an *equivalence oracle* that decides whether a given hypothesis monitor is indeed correct. The membership oracle can be implemented by computing the risk for a single trace as above and comparing it to a threshold, while the equivalence oracle can be implemented by checking for the absence of missed and false alarms. Practically, some minor modifications are necessary to embed our approach in AAL framework: We have to handle the finite horizon, resolve some traces that are not relevant for the monitor, and we combine conformance and equivalence queries to boost performance.

Putting it together. In Fig. 2, we outline our proposed framework to learn a monitor. SJ: TODO explain framework

Empirical evaluation. SJ: TODO summarise results of the empirical eval here.

Contributions. In summary, this paper provides a first framework to learn runtime monitors — encoded as finite automata — that are verified to be correct with respect to a given HMM. The main contributions are as follows: (1) We study monitor verification and show complexity-theoretic hardness of the underlying problem, even in an approximative setting. (2) We reduce the monitor verification to probabilistic model synthesis problems that exhaustively search for a counterexample. (3) We learn monitors by using the verifier above to answer equivalence queries in conjunction with active automata learning. (4) We demonstrate the feasibility and limitations of both the verifier and the learner on a series of benchmarks. Our prototype can handle HMMs with thousands of states, up to a hundred observations, and monitors with over 100 states.

[2] Alternatively, in Section **??**, we show that we can construct a product of an acyclic DFA accepting sets of traces and the HMM. However, this yields partially observable MDP, where we must search for memoryless policies, similar to a construction in [4]. This problem is equivalent to the synthesis problem and can therefore also be solved using Paynt [1].

2 Problem Statement

In this section, we recap standard notions on stochastic models and finite automata and introduce the formal problem statement.

Notation. Let X be a finite set. A distribution μ over X is a mapping $\mu\colon X \to [0,1]$ such that $\sum_{x \in X} \mu(x) = 1$. The set of all distributions over X is written $\Delta(X)$.

2.1 Models

In this section, we briefly introduce MDPs, which are important later, and HMMs which are crucial to the problem statement. Details can be found in [5].

Definition 1 (MDP). *A* Markov decision process *(MDP) is a tuple* $\mathcal{M} = (S, \iota, Act, \mathbf{P})$ *with a countable nonempty set* S *of* states*, the* initial state $\iota \in S$*, the* partial transition function $\mathbf{P}\colon S \times Act \rightharpoonup \Delta(S)$*.*

We use $Act(s) = \{a \mid \mathbf{P}(s,a) \neq \bot\}$ as the set of enabled actions. We assume that there are no deadlocks, i.e., for every $s \in S$, $Act(s) \neq \emptyset$. A path π in an MDP is a sequence $s_0 a_0 s_1 \ldots \in (S \times Act)^* \times S$, such that $a_i \in Act(s_i)$ and $P(s_i, a_i)(s_{i+1}) > 0$ for every $i \geq 0$. The set of (finite) paths is denoted by $\Pi^{\mathcal{M}}_{fin}$ and the set of paths of at most length h are $\Pi^{\mathcal{M}}_{h}$. If the MDP is clear from the context, the $\mathcal{M}$ is dropped.

A policy for an MDP is a function $\sigma\colon \Pi^{\mathcal{M}} \to Act$, which given a path through the MDP, gives a action. Given a policy, a MDP can be induced into a MC. Policy synthesis for an MDP for a property ϕ thus entails finding a policy for an MDP such that the induced MC entails ϕ.

A *Markov chain* (MC) is an MDP where $|Act(s)| = 1$ for every state $s \in S$. We simplify notation and write MCs also as tuple $(S, \iota, \mathbf{P})$ and write $\mathbf{P}(s)$ to refer to the unique distribution $\mathbf{P}(s,a)$. Paths in a MC can be simplified to be sequences of (only) states.

The probability measure $Pr^{\mathcal{M}}$ of an MC $\mathcal{M}$ is the unique probability measure following from the canonical σ-algebra associate with mc.

Condition probability on traces defined using Paths

Definition 2 (HMMs). *A (risk-labelled) HMM is a tuple* $(S, \iota, \mathbf{P}, Z, obs, r)$ *such that* $(S, \iota, \mathbf{P})$ *is a MC,* Z *is a finite set of observations, and* $obs\colon S \to Z$ *is the (deterministic) observation function*[3]*, and* $r\colon S \to \mathbb{R}_{\geq 0}$*,*

Notions such as paths are lifted from MCs. Furthermore, traces τ are sequences of observations, and the set of traces of a HMM is the language $\mathcal{L}(\mathcal{M})$ of the HMM. The set $\mathrm{Paths}(\tau)$ contains all paths that correspond to a trace τ, $\mathrm{Paths}(\tau) = \{\pi \in \Pi^{fin} \mid obs(\pi) = \tau\}$. SJ: Also needs things like the probability of a trace.

[3] We use determinstic observation functions for concise definitions. Non-deterministic observation functions can be expressed via a blowup of the HMM, see e.g. [**empty citation**]

Example 1.

Definition 3. *A deterministic finite automaton (DFA) is a tuple* $\mathcal{A} = (Q, \Sigma, \delta, \iota, F)$*.* Q *is a finite set of states,* Σ *is an alphabet,* $\delta \colon Q \times \Sigma \rightharpoonup Q$ *is a (partial) transition function,* ι *is the initial state, and* $F \subseteq Q$ *is a set of accept (or: final) states.*

Let ε denote the empty word. We lift the transition relation to words, which is defined recursively: $\delta^*(q, \varepsilon) = \varepsilon$ if $a \in \Sigma$ and $\delta^*(q, w \cdot a) = \delta(\delta^*(q, w), a)$.

The *language* $\mathcal{L}(\mathcal{A})$ of a DFA $\mathcal{A}$ is the set of all words accepted by the $\mathcal{A}$, $\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta^*(\iota, w) \in F\}$.

2.2 Formal Problem Statement

The monitor verification problem consists of two sub problems. The no false alarms problem, and the no missed alarms problem. Both of these sub problems entail finding the trace with either the minimum or maximum risk. We will discuss the missed alarm problem in the following sections. In section 3.4 we will adapt to the no false alarms problem.

The no missed alarms problem entails ensuring that all traces from the MC, which are not in the monitor, have at most a risk of λ.

Definition 4 (Risk of a trace). *Given a HMM* $\mathcal{M}$*. The risk of trace* $\tau \in \mathcal{L}(\mathcal{M})$ *is:*

$$R_r(\tau) := \sum_{\pi \in \Pi^h_{\mathcal{M}}} Pr(\pi \mid \tau) \cdot r(\pi_{\downarrow}).$$

The risk of a trace can be used to partition the set of all traces.

Definition 5 (Safe traces/Unsafe traces). *Given a HMM* $\mathcal{M}$*, a* safe threshold $\lambda_s \in \mathbb{R}$ *and an* unsafe threshold $\lambda_u \in \mathbb{R}$ *with* $\lambda_s \leq \lambda_u$*. A trace* $\tau \in \mathcal{L}(\mathcal{M})$ *with* $R(\tau) > \lambda_u$ *is an* unsafe trace *(for* $\mathcal{M}$ *and* λ_u*). Conversely, trace* τ *is a* safe trace*, if* $R(\tau) < \lambda_s$ *(for* $\mathcal{M}$ *and* λ_u*).*

In the definition above, we deliberately do not require $\lambda_s = \lambda_u$. In particular, by picking $\lambda_s < \lambda_u$, some traces are neither safe nor unsafe. We write $\mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$ to denote the set of safe traces of length at most h and $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u}$ for the set of unsafe traces of length at most h. We often omit the sub/superscripts.

Example 2.

We are now ready to define correct monitors.

Definition 6 (Monitor). *A DFA* $\mathcal{A}$ *is a monitor for HMM* $\mathcal{M}$ *if the alphabet for* $\mathcal{A}$ *coincides with the observations in* $\mathcal{M}$*.*

Definition 7 (Missed/False alarms). *Given a monitor* $\mathcal{A}$ *for HMM* $\mathcal{M}$*, a horizon* h*, and thresholds* $\lambda_s \leq \lambda_u \in \mathbb{R}$*, the set of* missed alarms *is* $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \setminus \mathcal{L}(\mathcal{A})$*. The set of* false alarms *is* $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s} \cap \mathcal{L}(\mathcal{A})$*.*

Fig. 3: Central example to the paper

We drop the sub/superscripts whenever possible. In terms of a classification problem, missed alarms can be seen as false negatives while false alarms are false positives.

Definition 8 (Correct monitor). *Given thresholds λ_s, λ_u and horizon h, a monitor $\mathcal{A}$ for $\mathcal{M}$ is* correct *if* $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset = \mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A})$.

Equivalently, a monitor is correct if it raises an alarm for every unsafe trace and for no safe trace:

Corollary 1 (Correct monitor). *A monitor $\mathcal{A}$ for HMM $\mathcal{M}$ with $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \subseteq \mathcal{L}(\mathcal{A}) \subseteq \Sigma^* \setminus \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$ is a correct monitor.*

Problem statements. This paper tackles the following problem statements: Given HMM $\mathcal{M}$, thresholds λ_s, λ_u and horizon h,

1. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there missed alarms, i.e., is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset$?
2. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there false alarms, i.e., is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}) = \emptyset$?
3. Find a correct monitor $\mathcal{A}$ for $\mathcal{M}$ w.r.t. λ_s, λ_u and h.

Problem statements 1 and 2 together allow checking whether a monitor is correct.

Remark 1. A correct monitor must exist, as $\mathbb{U}^{\leq h}$ is finite and thus regular.

We discuss our solution to the no missed alarms problem in section 3, we adapt the solution to the no false alarm problem in section 3.4.

Central example We now introduce the central example used in this paper. And explain how the monitor verification problem and verified monitor learning problem apply to it.

Example 3. Given the risk function assigning 1 to q_c and 0 to all other states, the traces $\tau_1 = dry, icy, icy$ and $\tau_2 = dry, icy$ have risks $\frac{19}{40}, \frac{4}{40}$ respectively. If λ is $\frac{1}{4}$ and the horizon h is 3, τ_2 is the trace with maximum risk not in the monitor. And its risk is below λ. Thus the monitor does not have an missed alarms.

Include verified monitor learning problem

3 Monitor Verification

In this section we will discuss our solution to the monitor correctness problem. We focus on the no missed alarms problem with traces of exactly the length of the horizon. We will solve this using policy synthesis of colored MDPs. At the end, we will generalize our construction to both the no false alarms problem and traces of length at most the horizon.

3.1 Colored MDPs

Policy synthesis for colored MDPs has seen recent successes[2]. We will reduce our problem into a colored MDP policy synthesis problem.

Definition 9 (Colored MDP). *Given an MDP $\mathcal{M}$, with states in S, a* colored MDP *is a tuple $\mathcal{M}^C = (\mathcal{M}, C, c\colon S \to C)$. C is the set of colors, and c maps states to colors.*

We will only consider policies for colored MDPs, which act equally for states of the same colors [4].

Definition 10 (Color consistent scheduler). *A scheduler σ for a colored MDP $\mathcal{M}^C$ with states in S is color consistent if for every two states s and s'*

$$c(s) = c(s') \Rightarrow \sigma(s) = \sigma(s')$$

We now present the theorem we focus on in this section.

Theorem 1. *Given a HMM $\mathcal{M}$ with a monitor $\mathcal{A}$, threshold λ_s, horizon h, and risk r, there exists a transformation f into colored MDPs, such that iff there exists a color-consistent policy for $f(\mathcal{M}, \mathcal{A}, \lambda_s, h, r)$ with $Pr > \lambda$ then there is a trace $\tau \in \mathsf{mA}(\mathcal{A})$*

Following from the transformation we will present we also have:

Corollary 2. *There is a one to one correspondence between a found τ and a color consistent policy.*

Remark that the no missed alarms problem contains several elements which are no longer present in the colored MDP. We will have to incorporate the monitor, horizon, observations, and risk. Furthermore, recall that the no missed alarms problem is stated in terms of a conditional probability on the trace. Policy synthesis cannot take conditional probabilities into account, thus we will also incorporate this in the transformation.

For clarity's sake, we describe our transformation in several steps. Only at the end do we move from HMMs to colored MDPs. Here we give a quick outline of the algorithm. In section 3.2 we will first move the monitor into the HMM. Secondly we will transform the HMM to both include the horizon and the risk. In section 3.3 we resolve the conditioning on traces, while also removing the observations from the model, by transforming into colored MDPs.

3.2 (A)CTP Problem

First, we show how the dual of the no missed alarms problem, the exists missed alarm problem, can be transformed into the simpler *Conditional trace probability*

[4] Colored MDPs with color consistent policies coincide with memoryless policies for POMDPs. However, since POMDPs often consider not only memoryless policies, we will use colored MDPs.

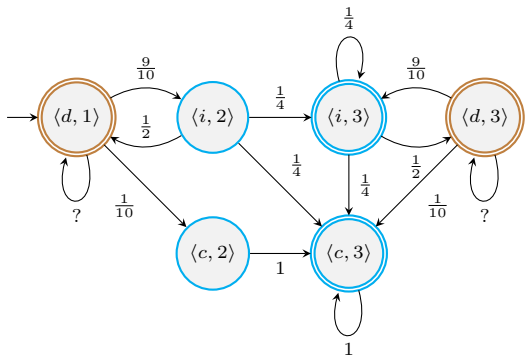


Fig. 4: The central example HMM and faulty monitor $\mathcal{B}$ transformed into the CTP problem. States are named with their HMM state, $\{d, i, c\}$ and their monitor state, $\{1, 2, 3\}$. States in the alarm states set A have been marked accepting.

(CTP) problem. Finally, we will further simply the problem by incorporating both the horizon and risk such that we are left with acyclic HMMs. We will discuss this intermediate problem as the Acyclic CTP (ACTP) problem.

The exists missed alarm problem, questions whether there is a trace τ such that $\tau \in \mathsf{mA}(\mathcal{A})$. Obviously, when such a trace exists, $\mathsf{mA}(\mathcal{A}) \neq \emptyset$, thus disproving the no missed alarms problem for $\mathcal{A}$.

CTP problem Before we combine the HMM and the monitor, we have to modify the monitor. Since we are looking for missed alarms, we are interested in all traces that are not in the monitor. We thus take the complement of the monitor.

We transform the HMM using the Product Markov Chain construction from Baier and Katoen [5]. It combines a DFA and a MC by taking the product of the states and using the obvious initial states and transition function. We extend this to HMMs, by letting the observation function observe the model states. This extension allows us to apply the Product Markov Chain construction to our model HMM and complemented monitor DFA, resulting in a product HMM. We also keep track of all resulting states for which the complement monitor accepts in a set of alarms states A of the product HMM.

The resulting product HMM can be seen as the model of a simplified version of the no missed alarms problem, the CTP problem.

Definition 11 (CTP Decision Problem). *Given an HMM $\mathcal{M}$ with states in S, a unary encoded horizon h, a set of alarm states $A \subseteq S$, a risk function $r\colon S \to \mathbb{R}^+$, and a threshold $\lambda_s \in (0, 1]$, find a trace $\tau \in \mathcal{L}(\mathcal{M})$ such that*

$$\sum_{\pi \in \Pi_H^{\mathcal{M}} \mid \pi_\downarrow \in A} Pr^{\mathcal{M}}(\pi \mid \tau) \cdot r(\pi_\downarrow) \geq \lambda$$

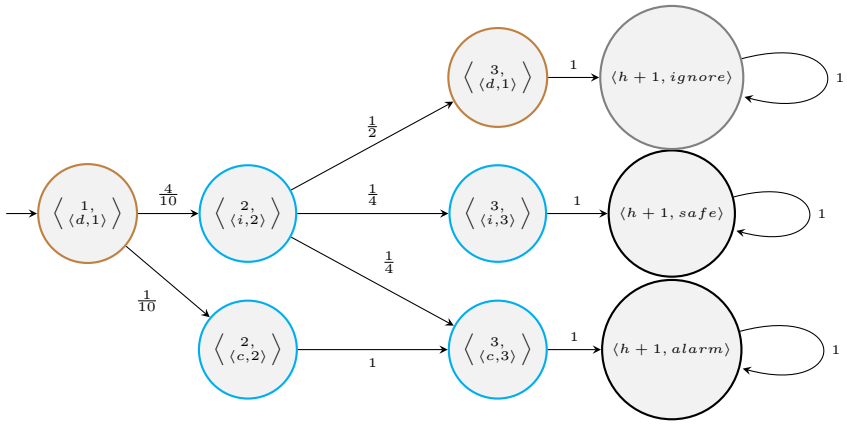


Fig. 5: ACTP, brown and cyan are dry and icy observations. Black is the new end observation, and gray is the new ignore observation. All states are named with the step i, model state s and monitor state j as $\left\langle \begin{smallmatrix} i, \\ \langle s,j \rangle \end{smallmatrix} \right\rangle$.

As we will be shown in section 5 the CTP problem is NP-complete. By using the above reduction to exists missed alarm problem, it is easy to show that the no missed alarms problem is co-NP-complete.

Our example HMM with monitor $\mathcal{B}$ from fig. 3 transformed into the CTP problem can be seen in fig. 4.

ACTP problem We further simplify the problem by combine the horizon, risk and the newly created alarm states set into the HMM. The resulting HMM can be seen as the model for the acyclic CTP problem.

Should I more properly introduce the CTP HMM with its contents? I do use them later on.

We will build up the ACTP HMM by creating its transition function $\mathbf{P}'$, all other elements of the ACTP HMM are easily derivable. To include the horizon into the ACTP HMM, we unroll the CTP HMM over the horizon h. The ACTP HMM has states in $\{1, \ldots, h\} \times S$. The ACTP transition function $\mathbf{P}'$ on non-horizon states behaves as expected:

$$\mathbf{P}'(\langle i, s\rangle, \langle i+1, q\rangle) = \mathbf{P}(s, q)$$

with $i \in \{1, \ldots, h-1\}$. The horizon states, $\langle h, s\rangle$ for $s \in S$, transition to one of three states, $\langle h+1, alarm\rangle$, $\langle h+1, safe\rangle$ or $\langle h+1, ignore\rangle$ state. Horizon states in A will transition to $\langle h+1, alarm\rangle$ and $\langle h+1, safe\rangle$ according to the normalized risk. Horizon states not in A transition to $\langle h+1, ignore\rangle$ instead.

Mention normalizing risk more prominently somewhere?

$$\mathbf{P}'(\langle h, s\rangle, \langle h+1, alarm\rangle) = \begin{cases} r'(s) & \text{if } s \in G \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbf{P}'(\langle h, s\rangle, \langle h+1, safe\rangle) = \begin{cases} 1 - r'(s) & \text{if } s \in G \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbf{P}'(\langle h, s\rangle, \langle h+1, ignore\rangle) = \begin{cases} 1 & \text{if } s \notin G \\ 0 & \text{otherwise} \end{cases}$$

The $\langle h+1, alarm\rangle$, $\langle h+1, safe\rangle$ and $\langle h+1, ignore\rangle$ states become sink states. The $\langle h+1, alarm\rangle$ and $\langle h+1, safe\rangle$ sates get the *end* observation, while the $\langle h+1, ignore\rangle$ state gets the *ignore* observation.

We add an $\langle h+1, ignore\rangle$ state, instead of redirecting all traces we don't care about to the $\langle h+1, safe\rangle$ state, to more easily modify the transformation for the no false alarms problem.

Remark 2. Note that we ignore all states in A that are not horizon states. However, since we only consider traces which are of length exactly the horizon, we only need to consider the alarm states which are encountered after horizon steps.

Using the above transformed HMM we can define the ACTP problem.

Definition 12 (Asyclic CTP problem). *Given an acyclic HMM $\mathcal{M}$ with observations Z, an alarm state a, and a threshold $\lambda \in (0, 1]$, is there a trace $\tau \in Z^\star$ such that $\sum_{\pi \in \Pi^{\mathcal{M}}} Pr^{\mathcal{M}}(\pi a \mid \tau) \geq \lambda$.*

In the ACTP HMM we no longer consider the risk of a trace, instead we only reason about the probability of reaching the alarm state. The ACTP problem thus contains a threshold between in the interval $(0, 1]$. We also no longer need to consider the horizon, as all traces consist of a horizon number of observation in Z and an infinite sequence of either *end* or *ignore* observations.

The ACTP transformation applied to the HMM from fig. 4 can be seen in fig. 5.

3.3 Reduction to Consistent Policy Synthesis

In the ACTP HMM we only have the observations and conditional probability on a trace left. To incorporate this into the model we will use one novel idea. Instead of transitioning to a state and observing it, we will allow the model to choose what observation to observe. We thus move away from HMMs and into MDPs. This has two results. Firstly, instead of talking about sequences of observations, instead we will consider sequences of actions taken. Secondly, any trace can be transformed into a policy of the MDP. We will first discuss how to create this MDP. Afterward, we show how to ensure how, traces and policies can be made equivalent by coloring the MDP.

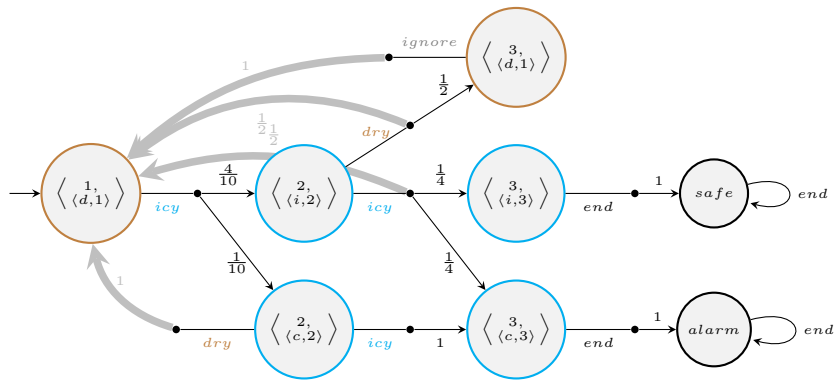


Fig. 6: The central example transformed into an mdp to be used as the model in the MDP policy synthesis problem. Every state has an action which contains only transitions to states which had the observation related to that action.

bad explanation

We construct the policy synthesis MDP where the transition function $\mathbf{P}''$ is defined as

$$\mathbf{P}''(s,z,q) := \begin{cases} \mathbf{P}'(s,q) & \text{if } obs(q) = z \\ \sum_{q' \in S \mid obs(q') \neq z} \mathbf{P}'(s,q') & \text{if } q = \iota \\ 0 & \text{otherwise} \end{cases}$$

Thus we transition normally to a state if the action and observation of the target state are equal, otherwise we set the transition probability to zero. All the remaining probability mass is redirected towards the initial state. We fully remove the ignore action and ignore state, as they should never be taken[5].

Should we give a reason why this is correct? And not normalizing the probabilities?

The above construction allows for conditioning on a trace τ by constructing a policy σ that selects the ith observation of τ in the state with step i.

Definition 13 (Trace consistent policy). *Given two sets S and Z and an MDP $\mathcal{M}$ with states in $\mathbb{N} \times S$, and actions in $Z \cup \{end\}$. Together with a trace $\tau \in Z^\star$. A* trace consistent policy *is of the form*

$$\sigma_\tau(\langle i, s \rangle) = \begin{cases} \tau^{(i)} & \text{if } i \leq |\tau| \\ end & \text{otherwise} \end{cases}$$

Corollary 3. *There exists a map $t(\sigma_\tau) = \tau$ which, given a trace consistent policy σ_τ finds its associated trace τ.*

However, when finding the maximal trace, we might not find a trace consistent policy. To fix this we color the MDP such that color-consistent policies coincide with trace consistent policies on our MDPs. As colors C, we take the set

[5] The ignore state was included in ACTP to ensure we had no deadlock states.

$\{1, \ldots, h\}$ of steps until the horizon, and as coloring function take the projection $c(\langle i, s\rangle) = i$. Given that all *end* states sinks, color consistent policies with the trace consistent policies.

Remark 3. All states gain all actions in $\mathbf{P}''$, we can later prune any actions for all states S^C of a color C when these actions only redirect to the initial state from S^C.

Now finding a trace with the conditional trace probability above λ_s coincides with finding a color consistent policy such that

$$Pr^{\sigma}(\Diamond\langle h+1, alarm\rangle) > \lambda_s$$

3.4 Adapting to No False Alarms and Smaller Traces

- Add end transitions to non horizon states.
- We want to minimize alarm probability, not possible.
- Instead maximize safe probability.

4 Learning Monitors

- Use MAT
- Filtering as MQ [7]
- Monitor correctness as EQ
- If not correct returns counter example trace

Design choices

- Limit filtering to horizon
- Use Random walk with MQ as pre-EQ

5 Complexity

Theorem 2. *Checking whether a monitor is correct with respect to a unary encoded horizon is coNP-complete.*

In fact, we study in this section the dual to this problem:

Definition 14.

NP-membership follows for the monitor co-verification as false alarms or a missed alarm (of length up to horizon) are witnesses. Verifying whether this trace is indeed a false alarm can be done in polynomial time by first checking whether the automaton accepts it and then computing the trace risk and comparing it to the thresholds. Each of these steps can be done in polynomial time. SJ: Maybe this result is so crucial that we should explain somewhere how this works in polynomial time.

For hardness, we show NP-hardness of the CTP problem from **??**. As a solution to the monitor co-verification problem suffices to solve the CTP problem, this implies NP harndess of the former.

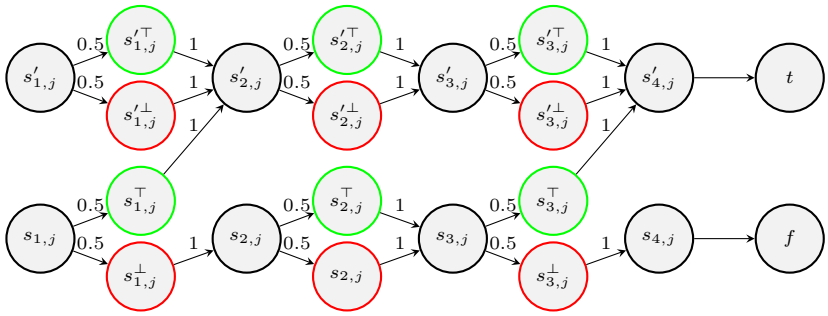


Fig. 7: The main gadget for the reduction of Lemma 1, demonstrated for the clause $x_1 \vee \neg x_3$ and variables x_1, x_2, x_3.

Lemma 1. *The CTP Decision Problem is NP-hard.*

SJ: What about the probability of a trace being zero?

The result holds even for $\lambda = 1$ and acyclic HMMs. We reduce the problem from 3CNF. We outline the construction below.

Consider a 3CNF formula $\varphi = \bigwedge c_1 \dots c_m$ over variables X, $|X| = n$, with clause $c_j = \ell_j^1 \vee \ell_j^2 \vee \ell_j^3$ and each literal $\ell_j^i \in \{x, \neg x \mid x \in X\}$. We transform this into a CTP instance with $\lambda = 1$ and an acyclic HMM $\mathcal{M}_\varphi$ with observations $Z = \{\#, \bot, \top\}$. The only state with positive risk is a state t. By construction, there is a trace with risk 1 iff there is a satisfying assignment to φ. Note that there is a trace with risk 1 iff there is a trace where all corresponding paths end in state t.

Before we give a formal definition of $\mathcal{M}_\varphi$, we give some intuition, supported by Fig. 7. We represent assignments $\alpha \colon X \to \{0, 1\}$ by traces through the HMM $\#\#\cdot\alpha(x_0)\cdot\#\cdot\alpha(x_1)\cdot\dots\cdot\#$. This trace matches with exactly m paths through the HMM, one for each clause. That is, intuitively, $\mathcal{M}_\varphi$ randomly selects a clause c_j with probability $1/m$ in the initial state and then 'evaluates' c_j with respect to an assignment α, by starting in $s_{1,j}$.

That is, a path through clause c_j 'reads' variable x_i in state $s_{i,j}$ and transitions to $s^{\top}_{i,j}$ and $s^{\bot}_{i,j}$, both with probability $1/2$. State $s_{i,j}$, $s^{\top}_{i,j}$ and $s^{\bot}_{i,j}$ have observations $\#, \top, \bot$, respectively. Thus, the conditional probability to reach $s^{\top}_{i,j}$ is $1/m$ if $\alpha(x_i) = \top$ and 0 otherwise. Then, if the literal x_i (or $\neg x_i$) occurs in clause c_j, we transition from $s^{\top}_{i,j}$ (or $s^{\bot}_{i,j}$, resp.) to $s'_{i+1,j}$, otherwise, we transition to $s_{i+1,j}$. The idea is that (conditionally) reaching state $s'_{i,j}$ with positive cconditional probability means that the partial assignment $\alpha(x_1), \dots, \alpha(x_i)$ satisfies clause c_j, while reaching $s_{i,j}$ with positive conditional probability means that the clause is either unresolved or unsatisfied given the partial assignment. Thus, t

We construct the following HMM $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ and set $\lambda = 1$:

- $S = \{s_{i,j}, s^{\bot}_{i,j}, s^{\top}_{i,j}, s'_{i,j}, s'^{\bot}_{i,j}, s'^{\top}_{i,j} \mid i \in \{1, \dots, n+1\}, j \in \{1, \dots, m\}\} \cup \{s_\iota\}$.
- $\mathbf{P}$ is given such that for all $i \in \{1, \dots, n\}, j \in \{1, \dots, m\}$:

- $\mathbf{P}(s_\iota, s_{1,j}) = \frac{1}{m}$,
- $\mathbf{P}(s_{i,j}, s^{\top}_{i,j}) = \mathbf{P}(s_{i,j}, s^{\bot}_{i,j}) = \frac{1}{2} = \mathbf{P}(s'_{i,j}, s'^{\top}_{i,j}) = \mathbf{P}(s'_{i,j}, s'^{\bot}_{i,j})$,
- $\mathbf{P}(s^{\top}_{i,j}, s'_{i+1,j}) = [x_i = \ell^k_j$ for some k$]$, $\mathbf{P}(s^{\top}_{i,j}, s_{i+1,j}) = [x_i \neq \ell^k_j$ for all k$]$,
- $\mathbf{P}(s^{\bot}_{i,j}, s'_{i+1,j}) = [\neg x_i = \ell^k_j$ for some k$]$, $\mathbf{P}(s^{\bot}_{i,j}, s_{i+1,j}) = [\neg x_i \neq \ell^k_j$ for all k$]$
- $\mathbf{P}(s'_{n+1,j}, t) = 1 = \mathbf{P}(s_{n+1,j}, f)$

– $c(s^{\top}_{i,j}) = (s'^{\top}_{i,j}) = \top$, $c(s^{\bot}_{i,j}) = (s'^{\bot}_{i,j}) = \bot$, $c(s_{i,j}) = c(s'_{i,j}) = \# = c(s_\iota) = c(t) = c(f)$.

–

Approximability Approximating the verification of monitors is also hard, in the sense that it is hard to approximate the maximal risk that a monitor admits.

The maximal risk problem is an optimization problem

Definition 15 (CTP Optimization Problem).

Lemma 2. *The CTP Optimization Problem is APX-hard.*

We show this by means of an L-reduction from MAX-3SAT, which is an inapproximable and APX-hard problem [**empty citation**]. The construction follows along the reduction for the NP-hardness.

6 Experiments

We build our tool in the python programming language on top of the model checker Storm [6] and the POMDP model checker PAYNT [2]. This tool includes both the ToVer verifier algorithm and the learning algorithm employing ToVer for equality queries.

TODO: Write further description of the tool.

Benchmarks We evaluate both the ToVer verifier and the learning algorithm. We evaluate the ToVer verifier using the following questions:

$\mathbf{Q}^1_{VM}$ How does the runtime of ToVer verifier depend on its parameters?
$\mathbf{Q}^2_{VM}$ How much does the ToVer verification depend on PAYNT?
$\mathbf{Q}^3_{VM}$ Which PAYNT strategy is the most efficient in the ToVer benchmarks?

Maybe not a full question?

We also evaluate the learning algorithm by looking at the following questions:

$\mathbf{Q}^1_{L}$ Does ToVer learning guarantee a better bound then learning using a sampling EQ?
$\mathbf{Q}^2_{L}$ How do the ToVer learning monitors and sampling learning monitors compare in size?
$\mathbf{Q}^3_{L}$ Do monitors learned without a horizon limit in MQ generalize to outside the horizon?

Again, maybe less relevant?

7 Related

- LTLf
- Predictors or so by Ziemek et al.
- MDPs as Distribution transformers
- Computing conditional probabilities (About reachability conditionals, not ending conditionals)
- Fairness related to semi-consistence and consistence.

8 Conclusion and Future Work

- Long traces (complexity and practical implementation).
- Unbounded traces (It woudl be great if we can add why this is hard)

References

[1] Roman Andriushchenko et al. "Inductive synthesis of finite-state controllers for POMDPs". In: *UAI*. Vol. 180. Proceedings of Machine Learning Research. PMLR, 2022, pp. 85–95.

[2] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[3] Dana Angluin. "Learning Regular Sets from Queries and Counterexamples". In: *Inf. Comput.* 75.2 (1987), pp. 87–106.

[4] Thom S. Badings et al. "CTMCs with Imprecisely Timed Observations". In: *TACAS (2)*. Vol. 14571. Lecture Notes in Computer Science. Springer, 2024, pp. 258–278.

[5] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[6] Christian Hensel et al. "The Probabilistic Model Checker Storm". In: *International Journal on Software Tools for Technology Transfer* 24.4 (Aug. 1, 2022), pp. 589–610. ISSN: 1433-2787. DOI: 10.1007/s10009-021-00633-z. URL: https://doi.org/10.1007/s10009-021-00633-z (visited on 09/02/2024).

[7] Sebastian Junges, Hazem Torfah, and Sanjit A. Seshia. "Runtime Monitors for Markov Decision Processes". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 553–576. ISBN: 978-3-030-81688-9. DOI: 10.1007/978-3-030-81688-9_26.

[8] Frits W. Vaandrager. "Model learning". In: *Commun. ACM* 60.2 (2017), pp. 86–95.